

Bases de Datos 2

Teoría 03

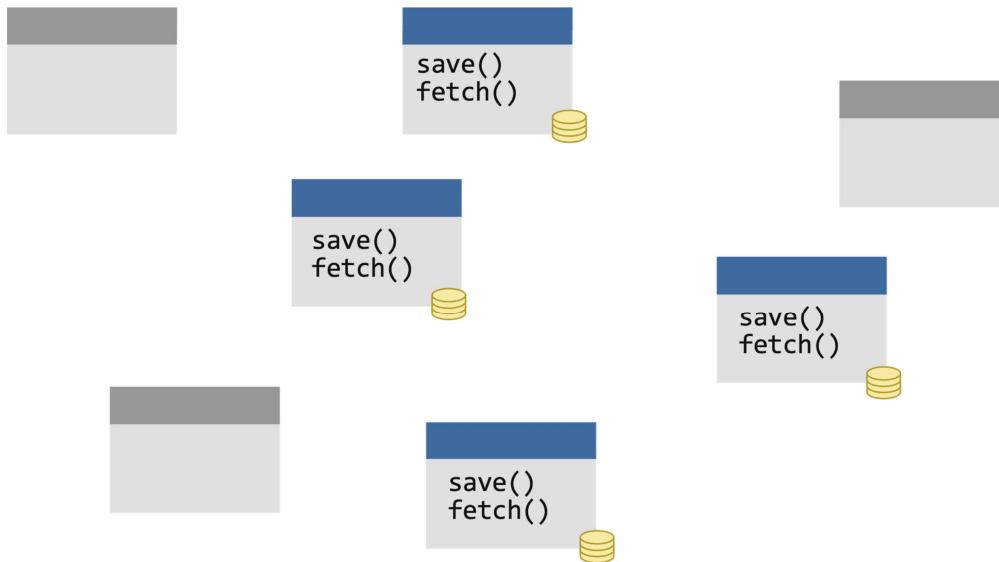
Federico Orlando

Patrones de Persistencia



Persistencia pre-ORM

¿Cómo se modelaría?



¿Cómo persistir un modelo de OO si no conociéramos los ORM?

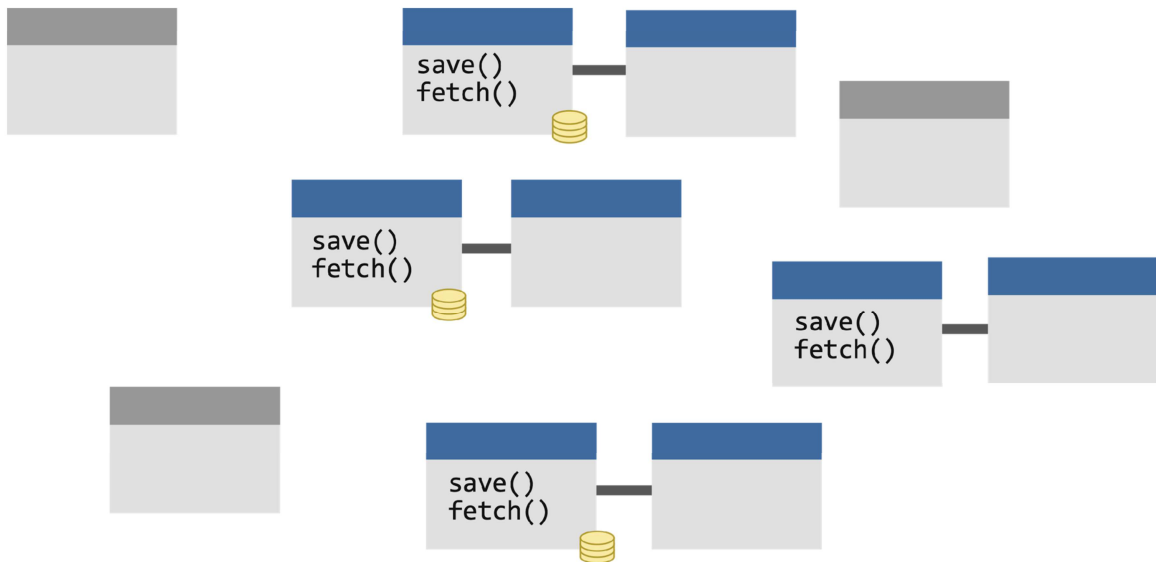
Caso naïve - cada clase se ocupa de su “guardar”, “obtener”, etc

Problemas:

1. Necesito una instancia de cada una para que funcione
2. Acoplamiento con la tecnología de persistencia

DAOs

Data Access Object



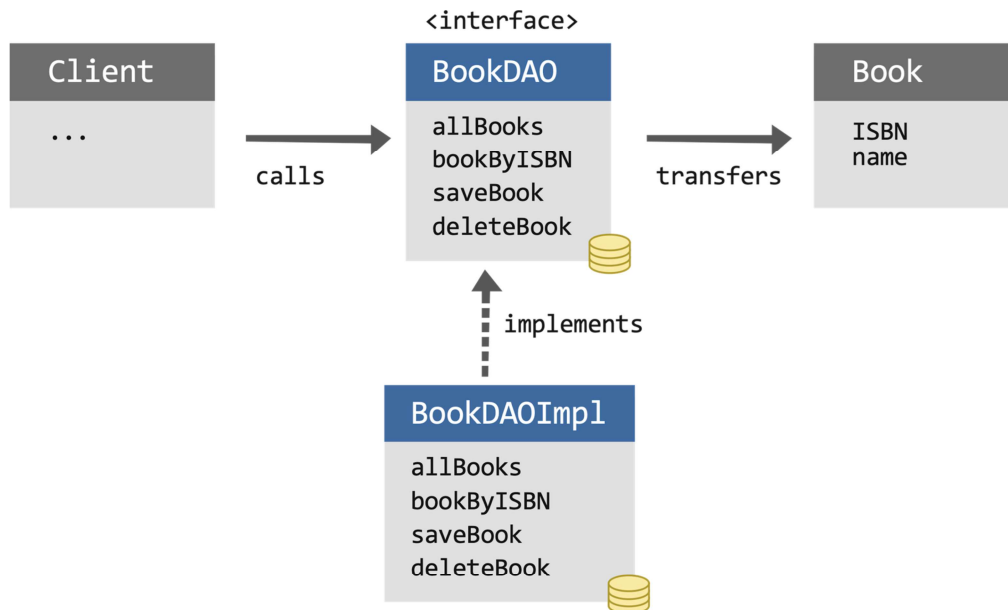
DAOs: establece que por cada objeto de dominio persistente debe existir una clase que persiste y recupera sus instancias.

Problemas:

1. Granularidad / relaciones entre DAOs de clases relacionadas
2. “Mudanza” del comportamiento de negocio a la capa de persistencia / el modelo queda anémico
3. Acoplamiento

DAOs

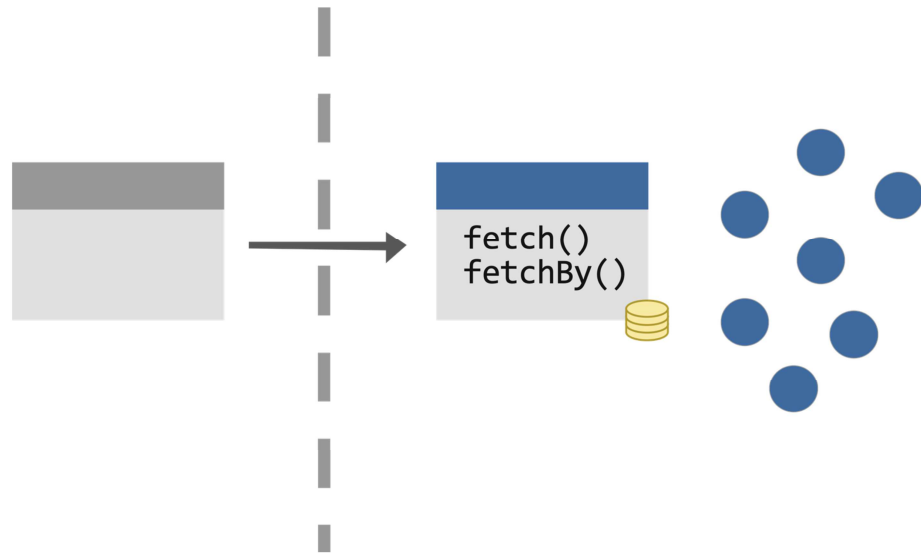
Esquema Típico



- Aparecen los ORMs ¿Dónde iría el ORM en este esquema?
- Se solía poner en el DAO, cosa que desaprovecha las capacidades de todo ORM.

Repository

Separando la BD del modelo



El patrón Repository ya no representa la **capa de persistencia**, sino la **colección** de objetos persistentes.

Se enfoca en el “fetch” - aprovecha los updates de ORM

Puede emular las colecciones de Smalltalk y sus filtros y usos de bloques (ej. `findByEmail`, `getUsersCount`)

Puede complicarse con el tipado estático - la implementación de puede volver repetitiva

Consultas



Lenguaje de Consulta

OQL

SQL	OQL
Select <atributos>	Select mensajes
From <tabla,tabs>	From Clase
Where <condiciones>	Where mensajes

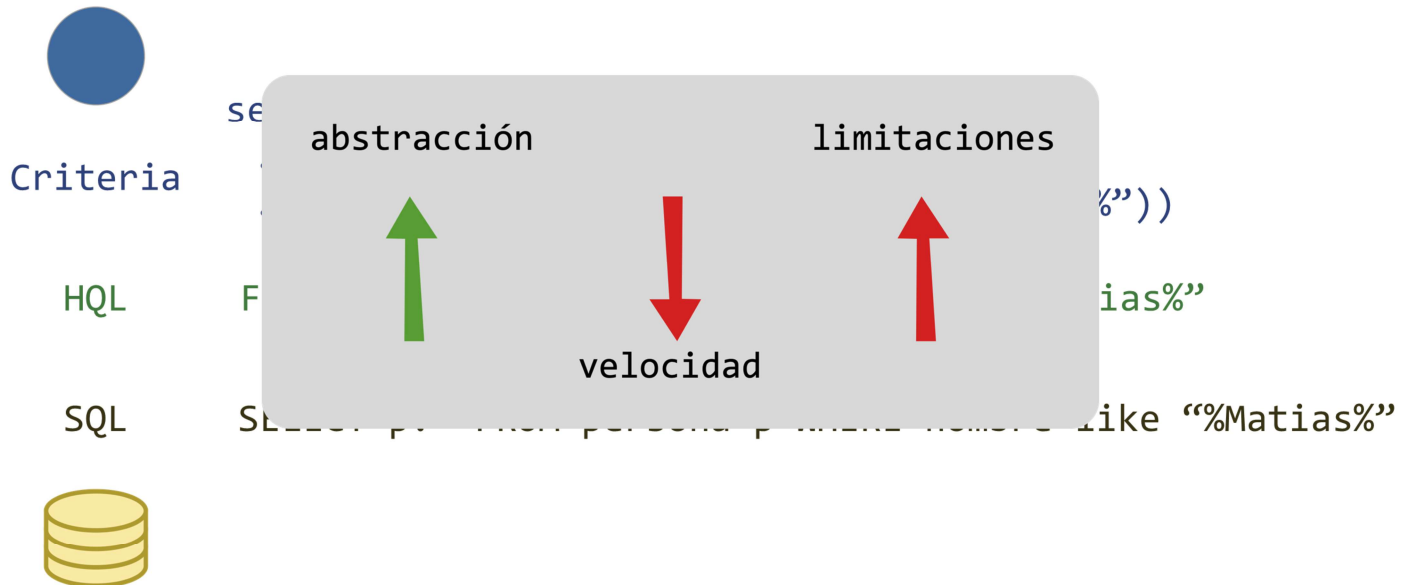
Ca. 2000 OMG (Object Management Group) publica un estándar para el acceso a las BBDD OO: **OQL (Object Query Language)**

Tomaron como base **SQL** - buscando mejorar la adopción

Se basa en la “extensión” de clase (Extent)

Lenguajes de Consulta

En Hibernate

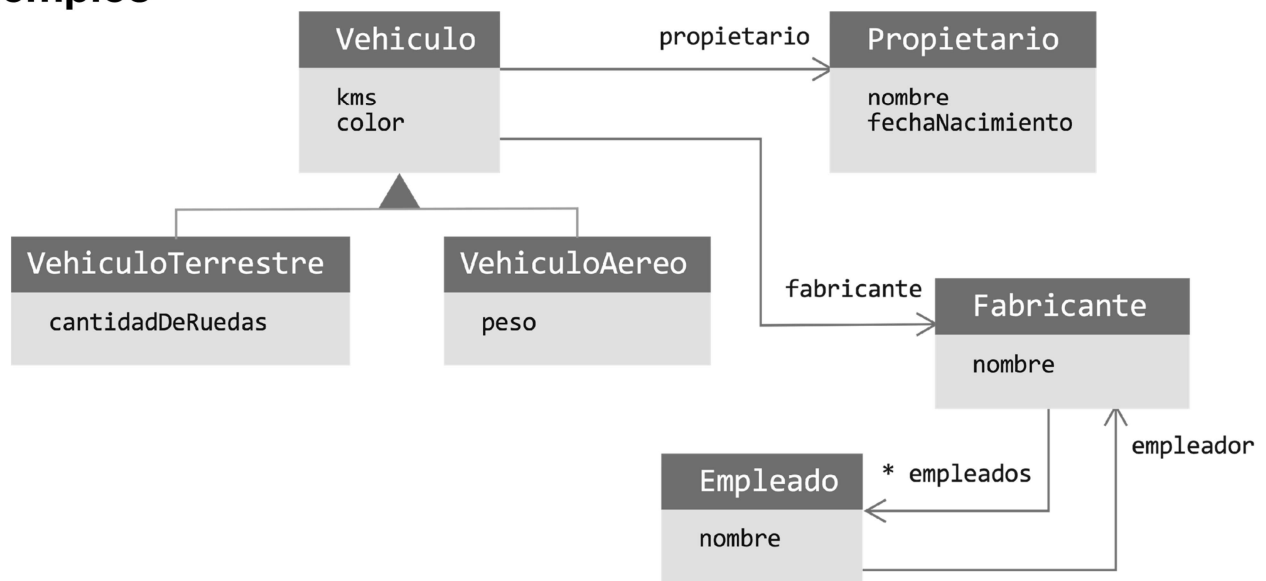


No es fácil implementar OQL, por lo que Hibernate ofrece:

- **SQL** es rápido, pero es un string (debugging complejo) y está atado a la BD 100% (tablas en vez de clases mapeadas). Puede formar objetos con la respuesta.
- **HQL** clases en el FROM. No puedo usar mensajes, solo atributos. Sigue siendo un string.
- **Criteria** se construye la query en objetos. Es más *debugger-friendly*. Más lento.

HQL

Ejemplos



HQL

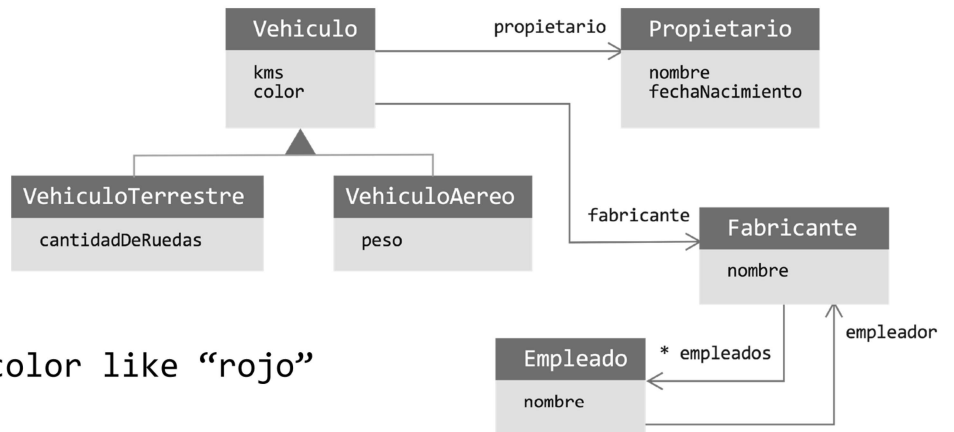
Ejemplos

from Vehiculo

from Vehículo v where v.color like “rojo”

```
select vehiculo
  from Vehiculo vehiculo join vehiculo.fabricante fabricante
  where fabricante.nombre like “Honda”
```

```
select v
  from Vehiculo v join v.propietario p join v.fabricante f
  where v.color like "rojo" and v.cantKilometros > 20000
    and f.nombre like "honda" and p.nombre like "juan"
```



HQL

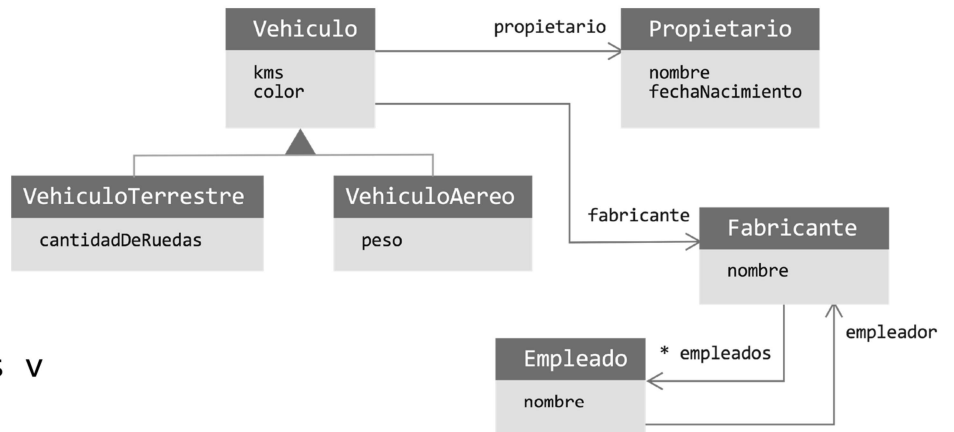
Ejemplos

```
select v
```

```
from VehiculoTerrestre as v
```

```
where v.color like "rojo" v.cantKilometros >
```

```
(select avg(v1.cantKilometros) from Vehiculo as v1 )
```



HQL

Tipos de Retorno

```
select reserva, cliente  
from Reserva reserva where ...
```



```
select new Cliente(nombre, fecha)  
from Cliente c  
where ...
```

También se pueden retornar arrays
de listas y más

HQL

Funciones de Agregación

`avg(...), sum(...), min(...), max(...)`

`count(*)`

`count(...), count(distinct ...), count(all...)`

HQL

Subconsultas

```
from Cat as fatcat  
where fatcat.weight > (select avg(cat.weight) from DomesticCat cat)
```

```
from Cat as cat  
where not exists ( from Cat as mate where mate.mate = cat)
```

```
from DomesticCat as cat  
where cat.name not in  
    (select name.nickName from Name as name)
```

Demo

SINGLE TABLE

```
@Entity
@Table(name = "empleados")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo_empleado", discriminatorType = DiscriminatorType.STRING)
public class Empleado {
    @Id
    @Column(name = "id_empleado")
    @GeneratedValue(strategy = IDENTITY)
    protected Long id;
```

✓ testGetEmpleadosAdministrativos() 73 ms

```
horariolab0.descripcion as descripc2_5_0_,
horariolab0.horaEntrada as horaentr3_5_0_,
horariolab0.horaSalida as horasali4_5_0_
from
HorarioLaboral horariolab0_
where
horariolab0_.id_horario=?
Tiempo de ejecución: 37 ms
```

✓ testGetAllEmpleados() 185 ms

```
horariolab0.descripcion as descripc2_2_0_,
horariolab0.horaEntrada as horaentr3_2_0_,
horariolab0.horaSalida as horasali4_2_0_
from
HorarioLaboral horariolab0_
where
horariolab0_.id_horario=?
Tiempo de ejecución: 146 ms
```

TABLE_PER_CLASS

```
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class Empleado {
    @Id
    @Column(name = "id_empleado")
    //@GeneratedValue(strategy = IDENTITY)
    @GeneratedValue(strategy = GenerationType.AUTO)
    protected Long id;
    @Column(nullable = false) 3 usages
    protected String legajo;
    @Column(nullable = false) 3 usages
    protected String apellido;
    @Column(nullable = false) 3 usages
    protected String nombres;
    @Column(nullable = false) 3 usages
    protected String genero;
    @Column(nullable = false) 3 usages
    protected String telefono;
    @Column(nullable = false) 3 usages
    protected Date fechaNacimiento;
    @ManyToOne 3 usages
    @JoinColumn(name = "horario_id")
    protected HorarioLaboral horario;
```

```
@Entity
//@DiscriminatorValue("VENDEDOR")
//@PrimaryKeyJoinColumn(name = "id_empleado")
@Table(name = "empleados_vendedores")
public class EmpleadoVendedor extends Empleado {

    private boolean esExclusivo; 3 usages
```

```
@Entity
//@DiscriminatorValue("ADMIN")
//@PrimaryKeyJoinColumn(name = "id_empleado")
@Table(name = "empleados_administrativos")
public class EmpleadoAdministrativo extends Empleado {
    private String Sector; 3 usages
    private boolean esProfesional; 3 usages
```

```
@Entity
//@DiscriminatorValue("OPERARIO")
//@PrimaryKeyJoinColumn(name = "id_empleado")
@Table(name = "empleados_operarios")
public class EmpleadoOperario extends Empleado{

    private String sector; 3 usages
```

✓ EmpleadoServiceTestCase (ar.edu.unlp.ir 275 ms)

✓ testGetAllEmpleados() 275ms

✓ 1 test passed 1 test total, 275 ms

horariolab0_.descripcion as descripc2_5_0_,
horariolab0_.horaEntrada as horaentr3_5_0_,
horariolab0_.horaSalida as horasali4_5_0_

from
HorarioLaboral horariolab0_

where
horariolab0_.id_horario=?

Tiempo de ejecución: 207 ms

✓ EmpleadoServiceTestCase (ar.edu.unlp.inf 67 ms)

✓ testGetEmpleadosAdministrativos() 67ms

✓ 1 test passed 1 test total, 67 ms

horariolab0_.descripcion as descripc2_5_0_,
horariolab0_.horaEntrada as horaentr3_5_0_,
horariolab0_.horaSalida as horasali4_5_0_

from
HorarioLaboral horariolab0_

where
horariolab0_.id_horario=?

Tiempo de ejecución: 33 ms

JOINED

```
@Entity
//SINGLE_TABLE
//@Table(name = "empleados")
//@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
//@DiscriminatorColumn(name = "tipo_empleado", discriminatorType = DiscriminatorType.STRING)
// JOINED
@Table(name = "empleados")
@Inheritance(strategy = InheritanceType.JOINED)
//TABLE_PER_CLASS
//@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public class Empleado {
    @Id
    @Column(name = "id_empleado")
    //GeneratedValue(strategy = GenerationType.IDENTITY)
    @GeneratedValue(strategy = GenerationType.AUTO)
    protected Long id;
    @Column(nullable = false) 3 usages
    protected String legajo;
    @Column(nullable = false) 3 usages
    protected String apellido;
    @Column(nullable = false) 3 usages
    protected String nombres;
    @Column(nullable = false) 3 usages
    protected String genero;
    @Column(nullable = false) 3 usages
    protected String telefono;
    @Column(nullable = false) 3 usages
    protected Date fechaNacimiento;
    @ManyToOne 3 usages
    @JoinColumn(name = "horario_id")
    protected HorarioLaboral horario;
```

```
1
@Entity
//@DiscriminatorValue("ADMIN")
//JOINED
@Table(name = "empleados_administrativos")
@PrimaryKeyJoinColumn(name = "id_empleado")
//TABLE_PER_CLASS
//@Table(name = "empleados_administrativos")
public class EmpleadoAdministrativo extends Empleado {
    private String Sector; 3 usages
    private boolean esProfesional; 3 usages
```

```
✓ EmpleadoServiceTestCase (ar.edu.unlp.in) 73 ms ✓ 1 test passed 1 test total, 73 ms
  ✓ testGetEmpleadosAdministrativos() 73 ms
    from
      HorarioLaboral horarioLab_
    where
      horarioLab0_id_horario=?
    Tiempo de ejecución: 36 ms
```

```
✓ EmpleadoServiceTestCase (ar.edu.unlp.in) 173 ms ✓ 1 test passed 1 test total, 173 ms
  ✓ testGetAllEmpleados() 173 ms
    from
      HorarioLaboral horarioLab_
    where
      horarioLab0_id_horario=?
    Tiempo de ejecución: 126 ms
```